

Analysis of software development methodologies from process-centric models to emerging perspectives

Project Background

Team Charter

The team consists of four members: Sun Weichao, Wang Bohong, Chen Xianzhe, Liu Haoyang.

1. Sun Weichao — Team Leader / Game Designer / Scrum Coordinator
2. Wang Bohong — Game Designer / Scrum Coordinator
3. Chen Xianzhe — Game Designer / Scrum Coordinator
4. Liu Haoyang — Game Designer / Scrum Coordinator

The project is an educational game for kindergarten students. The games will be created using KOPO, while Figma will be used as the shared collaboration tool for planning, visual layout, and team communication.

All team members share similar relevant skills and responsibilities, including basic game design, game creation, Figma collaboration, task discussion, peer feedback, and progress coordination. Each member will create three game themes.

Analysis of team profile: The team profile is suitable for an Agile-style project because the team is small, flexible, and self-managed. The main challenge is maintaining consistent design, clear learning goals, and regular coordination. Shared Scrum coordination, Figma collaboration, and regular progress checks will help the team manage these risks.

Project Description

Project background: The project is an educational game for kindergarten students. It focuses on play-based learning through simple activities such as ordering, sorting, comparing, or basic English learning tasks. KOPO will be used to create the games, while Figma will support shared planning, visual layout, and team collaboration.

Project goals and milestones: The main goal is to create simple, clear, and age-appropriate educational games. During the Alpha stage, each team member will create three game levels or themes, giving the group twelve in total. For the Beta and Final stages, the team will review the initial outcomes, refine the selected game direction, and improve the final game themes.

Project contract and interviews: The project contract and interview activities will help the team understand user needs, learning goals, and expected game functions. Feedback will be used to improve the game design.

Team responsibilities: Each team member should create three game themes, contribute to Figma collaboration, share progress, provide peer feedback, and help maintain consistent design and learning objectives.

Section 1: Software Development Approaches

With societal development, the importance of software engineering continues to rise. Software engineering has a history of managing complexity, scale, and uncertainty. The two most distinctive models among them are the Waterfall model and the Agile approaches. To understand the current industry landscape, it is essential to delve into the different philosophies, historical contexts, and underlying logics of these models.

The waterfall model was created in the early 1970s. It emerged as a response to the "software crisis". Due to software projects at the time generally faced problems such as cost overruns, unreliable products, and a lack of engineering rigor, early theorists drew inspiration from established disciplines such as civil engineering, attempting to introduce an orderly and predictable control framework (Sommerville, 2016). The most basic operating assumption of the waterfall model is that software requirements are stable, predictable, and can be fully extracted and solidified in the very early stages of a project. Therefore, the development lifecycle is divided into clear, linear, and non-overlapping phases, including software specification (requirements engineering), software design, software implementation (coding), software verification (component, system, and acceptance testing), and long-term software evolution (maintenance). In a traditional waterfall process, each activity must be strictly documented and fully closed before the project can move to the next phase. These progresses are measured against a predefined baseline plan. This structured linear logic assumes a high degree of determinism. This formalization remains highly relevant in security critical areas. However, the business market rarely offers perfect parameters, such as allowing customers to fully articulate their requirements before writing the first line of code. This shows the fatal flaw of the waterfall model when dealing with dynamic business environments.

As the global economy goes through digital transformation, the heavy management overhead, excessive documentation, and slow market response resulting from planning-oriented processes. These problems have sparked strong dissatisfaction among practitioners. This led to the creation of the Agile approaches (Beck et al., 2001), it completely changed the underlying assumptions of software engineering. Agile approaches abandon static predictability. People believe that change is an inevitable reality brought about by the continuous evolution of the business environment and market demands. The operating philosophy of software projects has shifted from "predicting change" through extensive upfront planning to "tolerating change" through incremental, interwoven development. Agile approaches not only divide projects into linear phases, but also break down software incrementally into fully functional parts. It can also iterate to allow features to

be gradually built, understood and refined over time, and quickly validated with continuous stakeholder involvement.

Under these approaches, The Scrum framework uses empirical process control to put agile principles into practice. It relies heavily on three core principles: transparency, inspection, and adaptation (Włodarczak, 2024). In Scrum, work is organized into short, fixed-cycle closed loops called "sprints". The framework clearly defines three roles: the Product Owner has a product backlog and is responsible for maximizing product value; the Developers have cross-functional skills and are committed to delivering high-quality product increments that meet the "definition of completion" in each sprint; and the Scrum Master acts as a coach to ensure team effectiveness and remove systemic obstacles. Requirements are no longer locked into lengthy software requirements documents, but are presented as a dynamically sorted product backlog and described in the form of lightweight "User Stories". In addition, through events such as sprint planning meetings, daily stand-up meetings, sprint review meetings, and sprint retrospective meetings, Scrum teams can continuously monitor progress, collect immediate customer feedback, and dynamically adjust development plans. The focus shifts from small things to assessing truly unresolved problems.

Section 2: Critical analysis of strengths, limitations, and failure

While both the waterfall model and agile approaches have unique structural advantages, an objective examination of their real-world applications reveals that both have obvious limitations. Although the software industry as a whole has shifted to agile approaches on a large-scale over the past two decades, software projects worldwide still frequently encounter serious cost overruns, schedule delays, and the inability to meet core user needs. These issues lead to a core reality: no matter how flexible a process model is, it cannot solve the inherent complexity of software engineering on its own.

The limitations of the waterfall model are well known because of its rigid sequence. During the development planning process, because software verification activities occur at the final stage of the linear lifecycle, this means that the mismatch between the engineered product and the customer's actual needs is often dangerously exposed only in the very late stages of the project. The cost and time of having to remake previous stages to adapt to later changes in requirements are extremely high. Furthermore, because tangible physical software cannot be delivered before the final stage, customers are forced to assess progress based on abstract textual specifications rather than running code. Due to a lack of early visibility, delivered systems, even if technically fully compliant with initial specifications, are often rejected by users, such as low user acceptance or changes in the market environment.

However, while agile approaches reduce the risk of problems arising later by interweaving specification, development, and testing, it also introduces structural drawbacks that cannot be ignored. First, because agile prioritizes working code over extensive paper documentation, the lack of significant and rigorous early documentation makes it difficult for management to accurately measure long-term progress at a macro level. Secondly, because the functions are rapidly and incrementally stacked without rigorous upfront architectural

design, the underlying internal design of the software is prone to gradual degradation over time. If development teams are unwilling to invest significant time and resources in continuous refactoring (rewriting internal code structures to improve maintainability without altering external behavior), then integrating future changes will become increasingly difficult, fragile, and costly. Finally, highly technical practices like "Test-first Development" require extremely strong team discipline; once developers sacrifice test quality under external deadlines, the reliability of the software will lose quickly.

So, why do professional software projects still fail even when using mature agile frameworks? There are three important reasons to explain this. Firstly, the trap of complexity and scale. Modern software is too large and too complex, such as AI and cloud computing, and it has to cope with complex corporate environments. Scrum is only used by small teams and cannot solve the major software technology and architectural challenges of large projects. Second, Cargo Cult Agile. Many companies only superficially learn Agile without truly understanding its principles. For example, leaders force teams to deliver results within a limited time. Agile becomes mere formalism. Third, the issue of individual ethics. Good software engineering requires the effort of everyone on the team. If everyone is dishonest and lazy, for example, only producing a half-finished product as a demonstration or trying to please management, then the project will certainly not succeed.

Section 3: Engagement with contemporary perspectives ("what comes next")

As the limitations of a purely process-centric approach become increasingly apparent, the modern software engineering profession is beginning to face some challenges. Is it really enough to just adhere to certain development frameworks, such as Scrum? A new consensus is appearing in the industry that software engineering practices must move beyond complexly management process structures and evolve towards a more integrated, human-centered, design-oriented cognitive model. Today's software practices are shifting from a focus on "how to manage tasks" to a deeper understanding of "how to explore problems" and "how to sustainably architect complex solutions." (Brooks, 1987).

In this development, a major cutting-edge advancement is the deep integration of "Design Thinking" directly with software engineering methodologies, thereby perfectly linking the "Problem Space" and the "Solution Space" early in the project. Traditional agile approaches often presuppose that there is already a set of clear, high-value user stories in the product backlog waiting for the team to execute. But how can a team uncover a client's true needs when they cannot clearly speak their pain points and requirements? This is the pain point that design thinking addresses. This is a non-linear, highly flexible cognitive methodology; design thinking uses five mindset models to manage requirements. First is Empathise. Engineers use divergent thinking, conducting open-ended customer interviews and in-depth preliminary research, to thoroughly understand the user's real-world environment and uncover the operational constraints and pain points hidden beneath the daily workflow. Second is Define. Using convergent thinking, the large of notes captured by team members in meetings are collaboratively summarized using affinity diagrams, thereby distilling practical core problem statements and transforming them into concrete user stories. Third is

Ideate. Use divergent thinking, employing tools such as brainstorming, mind mapping, or structured sketches, to challenge existing business assumptions and reframe core issues from multiple perspectives to incubate innovative ideas. Forth is Prototype. Using convergent thinking, prototypes of varying fidelity are built by exploring and creating tangible representations of ideas. This means moving from extremely low-cost and rudimentary Lo-fi paper prototyping to mid-fidelity wireframes focusing on structural layout, and finally to high-fidelity mockups, to intuitively explore interface layout, data flow, and system direction. Fifth is Test. Deploy the developed prototype directly to the target user group and evaluate it using structured user interaction methods such as the "5-act Interview". This is cheap and quick to verify the system's usability, intuitiveness, and potential value before the team has written a single line of real code.

This novel design-oriented perspective reshapes the "requirements engineering" traditionally part of the waterfall model into a continuous, creative, and highly experimental cognitive process. By conducting extensive hypothesis testing using cheap prototypes before coding, teams can safely experiment and fail. This successfully transforms software engineering from a "code execution factory" into a dynamic "knowledge learning system."

Section 4: Reflection

After a deep reflection on the various software engineering methodologies discussed in this report, it becomes clear that the success or failure of managing complexity and eliminating uncertainty in software projects does not depend on perfect processes and diagrams, but rather heavily on the quality of human teamwork, deliberate coordination mechanisms, and pragmatic modeling capabilities. The process framework only provides the foundation. The team's psychological and operational motivation within it is what breathes life into that foundation and increases the likelihood of success.

Furthermore, the common practices highly emphasized in agile and design thinking methodologies, such as pair programming, code reviews, and cross-functional whiteboard brainstorming. These are often valuable.

Finally, modeling plays an important cognitive bridging role in managing the uncertainties of the software lifecycle. Software engineering has never been only a purely hardcore technology about computer programs, but rather a comprehensive socio-technical discipline that deeply intertwines human organization, the art of design, and engineering science.

Reference (Use APA style)

Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R., Mellor, S., Schwaber, K., Sutherland, J., Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org/>

Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4), 10-19. <https://www.cs.unc.edu/techreports/86-020.pdf>

Sommerville, I. (2016). *Software engineering*. Pearson. <https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf>

Wlodarczak, P. (2025). *Agile software development* (1st ed.). CRC Press, Taylor & Francis Group. <https://doi.org/10.1201/9781003301707>

Assessment 1: Rubric

Rubric Criteria	HD (85–100%)	D (75–84%)	C (65–74%)	P (50–64%)	F (0–49%)
Understanding of software development approaches 6%	Demonstrates a sophisticated and accurate understanding of Waterfall, Agile, and post-Agile perspectives, including underlying assumptions and historical context.	Demonstrates a strong understanding of Waterfall, Agile, and emerging perspectives, with minor gaps or simplifications.	Demonstrates a sound understanding of Waterfall and Agile, with limited or surface engagement with what may come next.	Demonstrates a basic understanding of software development approaches, with significant omissions or inaccuracies.	Demonstrates little or no understanding of software development approaches.
Critical analysis of strengths, limitations, and failure 9%	Provides insightful, well-reasoned analysis of why software projects fail despite process models, critically evaluating the limitations of Agile and Waterfall.	Provides a clear and reasoned analysis of limitations and failure, with some critical depth.	Provides descriptive discussion of strengths and limitations, with limited critical evaluation.	Provides largely descriptive or simplistic discussion with minimal analysis.	Provides little or no analysis or relies on unsupported assertions.
Engagement with contemporary perspectives (“what comes next”) 6%	Engages deeply with contemporary or emerging ideas beyond traditional Agile, demonstrating	Engages clearly with at least one contemporary perspective, showing understanding and some	Refers to emerging ideas but with limited depth or critical engagement.	Mentions future or alternative approaches superficially or unclearly.	Does not engage with perspectives beyond Waterfall and Agile.

	original insight and thoughtful interpretation.	critical reflection.			
Use of evidence and literature 6%	Integrates high-quality academic and professional sources effectively to support arguments, demonstrating strong synthesis.	Uses relevant sources appropriately to support arguments, with good integration.	Uses sources adequately, though integration may be uneven or largely descriptive.	Uses limited or weak sources or relies heavily on unsubstantiated claims.	Uses little or no appropriate evidence, or sources are irrelevant or absent.
Clarity, structure, and academic communication 3%	Writing is clear, coherent, and well-structured; arguments flow logically; APA 7 referencing is accurate and consistent.	The writing is clear and well-structured, with minor issues; APA 7 referencing is mostly correct.	The writing is generally clear but may lack cohesion; APA 7 referencing contains some errors.	The writing is difficult to follow in places; the structure and referencing are inconsistent.	Writing is unclear or poorly structured; referencing is incorrect or absent.